Research — Cartridges

Nov 4, 2026 · @Tim

Summary

Cartridges is end-of-life: its Codeberg repo was archived and Flathub flagged it EOL on 2026-09-03, with v2.13.1 (2025-09-24) as the last release. It is the closest existing app to ours, a Python + GTK4 + libadwaita "launcher over launchers" that imports games from other stores and launches them through their URI handlers. An unreleased `rewrite` branch (last commit 2026-04-05) adds a `Gtk.GridView`, `Adw.Sidebar`, collections and libmanette gamepad navigation; the README names it the best base for a fork but warns it is not fully functional.

For us it is the best reference for importer detection paths, the 2:3 cover pipeline and a GObject list-model stack, and its retirement leaves a gap in the GNOME ecosystem that our app can fill.

Project facts

Cartridges is GPL-3.0-or-later Python on GTK4 + libadwaita + Blueprint, archived since 2026-09-03.

Field	Value
App ID	`page.kramo.Cartridges`
Latest release	v2.13.1, 2025-09-24; rewrite branch never released
Status	Archived on Codeberg and marked end-of-life on Flathub, 2026-09-03
Canonical repo	codeberg.org/kramo/cartridges; GitHub kramo/cartridges is a mirror; never moved to GNOME GitLab
Licence	GPL-3.0-or-later
Language	Python (about 97%), Meson; rewrite needs Python 3.13+, strict pyright, ruff

Field	Value
Toolkit	`gtk4 >= 4.15`, `libadwaita-1 >= 1.6`; Flathub build on org.gnome.Platform 49; rewrite targets the runtime master branch and uses `Adw.Sidebar` and `Adw.WrapBox`
UI files	Blueprint (`.blp`) throughout
Python deps (Flatpak)	pyyaml, pillow, requests (list may be incomplete)
GNOME Circle	Badge added 2023-05-23; no longer in the current Circle list on apps.gnome.org (reason unconfirmed)
Translation	Hosted Weblate; commits until 2026-08-07
People	kramo (Laura Kramolis); rewrite mostly by Jamie Gravendeel and Zoey Ahmed; stable importer pipeline by Geoffrey Coulaud
Platforms	Linux (Flathub), Windows, macOS; rated 5/5 for mobile fit by LinuxPhoneApps

UI and navigation

Stable Cartridges is a sidebar of sources beside a 2:3 cover grid, with a full-page details view; the rewrite moves to `Adw.Sidebar`, `Gtk.GridView` and collections. Details below come from the `.blp` files on `main` and `rewrite`.

Stable v2.13 window

- `Adw.ApplicationWindow` (minimum 360×100, default 1170×795) → `Adw.ToastOverlay` → `Adw.NavigationView` with three pages: library, hidden library, game details.
- Library page is an `Adw.OverlaySplitView` (sidebar 0.2 of width). The sidebar is a `navigation-sidebar` `Gtk.ListBox`: All Games and Added rows with counts, then an Imported heading and one row per source with icon and count.
- Header bar: sidebar toggle, a + menu (Add Game, Import), main menu, search toggle. Search is a `Gtk.SearchBar` with the entry clamped to 500 px and key capture on the page.
- Grid is a homogeneous `Gtk.FlowBox` (12 px spacing, 15 px margins, no selection) using filter and sort functions, not a `GridView`.

- Breakpoint at 564 px collapses the split view and stacks the details page vertically.

Game tile

- A `card` button: 200×300 (2:3) `Picture` with an `Adw.Spinner` while loading, ellipsized title below.
- On hover, two crossfading revealers show a circular OSD play button (top left) and a ⋯ menu (Edit, Hide or Unhide, Remove) at top right.
- Clicking the cover opens details; the "Cover Image Launches Game" setting swaps that so the cover plays and the small button opens info.

Details page and dialogs

- Full-window blurred cover as background; 200×300 cover, `title-1` title, developer, relative Added and Last played dates.
- Opaque pill Play button plus circular Edit, Hide, Remove and a "Search on…" menu (IGDB, SteamGridDB, ProtonDB, PCGamingWiki, Lutris, HowLongToBeat).
- Add/Edit is a 480 px `Adw.Dialog`: cover with New and Delete cover buttons, `Adw.EntryRow`s for Title, Developer and Executable (file chooser plus a help popover on the format).
- Hidden games live on their own navigation page with their own grid and search (Ctrl+H).
- Sort options: A–Z, Z–A, Newest, Oldest, Last Played (default).
- Empty states use `Adw.StatusPage`: No Games (with an Import pill), No Games Found, No Hidden Games.

Preferences (`Adw.PreferencesDialog`, searchable, three pages)

- General: exit after launching, cover launches game, high-quality images, and a destructive Remove All Games `Adw.ButtonRow`.
- Import: import on startup, remove uninstalled games, then one `Adw.ExpanderRow` per source with an enable switch, an install-location folder chooser and sub-toggles (for example Heroic: Epic, GOG, Amazon, Sideloaded).
- SteamGridDB: API key, use SGDB, prefer over official images, prefer animated images, Update Covers button.

Feedback and shortcuts

- Toasts: "{game} launched"; hide, unhide and remove toasts carry Undo (Ctrl+Z); a summary toast after import.

- Import runs in a non-closable `Adw.Dialog` with a progress bar; errors collect into one `Adw.AlertDialog` with a Preferences button.
- Shortcuts: Ctrl+Q, Ctrl+, Ctrl+N, Ctrl+I, Delete, F9, Ctrl+H, Alt+Up, Alt+Home, Ctrl+F, Ctrl+Z, F10, Ctrl+W. Delete fires even while typing in a field (issue #337).

Rewrite branch window

- `Adw.Sidebar` (224 px, collapses at 730 px) with sections All Games, Sources and Collections (edit, remove, New Collection).
- Search entry is the header-bar title widget, with a Sort & Filter menu beside it.
- Content is an `Adw.ViewStack` switching the grid and empty states. The grid is a `Gtk.GridView` (up to 12 columns, single-click activate) over a filter model combining collection, name/developer search and hidden filters.
- Tile shows a centred Play pill at the bottom of the cover and a top-right menu with a collections checklist.
- Details and editing merge into one navigation page; the app forces dark style.

Importers

Cartridges detects nine sources by probing native and Flatpak paths in order, and launches every store game through that store's own URI handler or CLI. "Host" below means the host XDG directories read from inside the sandbox.

Source	Detection paths (files required)	Metadata	Cover	Launch
Steam	`~/.steam/steam`, `$XDG_DATA/Steam`, `~/.var/app/com.valvesoftware.Steam/data/Steam` (`steamapps/libraryfolders.vdf`, `appcache/librarycache/`)	`appmanifest_*.acf` per library; installed if `stateflags & 4`. Stable calls the Store API for developer and to blacklist non-games; rewrite parses binary `appinfo.vdf` offline (type, developer, last played)	`librarycache/<appid>/library_600x900.jpg`	`xdg-open steam://rungameid/<appid>`
Lutris	`~/.var/app/net.lutris.Lutris/data/lutris`, `$XDG_DATA/lutris`, host (`pga.db`)	Copies the DB to a temp dir, then SQL: id, name, slug, runner; `.hidden` category maps to hidden	`coverart/<slug>.jpg`	`xdg-open lutris:rungameid/<id>`

Source	Detection paths (files required)	Metadata	Cover	Launch
Heroic	`~/.config/heroic`, host, `~/.var/app/com.heroicgameslauncher.hgl/config/heroic`	Sub-sources: Epic (`store_cache/legendary_library.json` + `legendaryConfig/legendary/installed.json`), GOG (`gog_library.json` + `gog_store/installed.json`), Amazon (`nile_library.json` + `nile_config/nile/installed.json`), sideload (`sideload_apps/library.json`)	`images-cache/<sha256 of art URL>`	`xdg-open heroic://launch/<runner>/<app_name>`
Bottles	`~/.var/app/com.usebottles.bottles/data/bottles`, `$XDG_DATA/bottles` (`library.yml`, `data.yml`)	Library entries; custom bottle path from `data.yml`	`<bottle>/grids/<thumbnail>`	`xdg-open bottles:run/<bottle>/<game>`
itch	`~/.var/app/io.itch.itch/config/itch`, `~/.config/itch` (`db/butler.db`)	SQL join of caves and games	Downloaded from `cover_url`	`xdg-open itch://caves/<cave>/launch`
Legendary	`~/.config/legendary` (`installed.json`, `metadata/`)	Skips DLC; developer from metadata	Epic tall box-art URL	`legendary launch <app>`
RetroArch (stable only)	Flatpak and native `retroarch.cfg`	`.lpl` playlists; core per item	`thumbnails/<playlist>/Named_Boxarts/`	`retroarch -L <core> <rom>`
Flatpak apps (stable only)	`/var/lib/flatpak/exports`, `~/.local/share/flatpak/exports`	`.desktop` files in category Game; skips known launchers unless enabled	App icon on a blurred 2:3 card	`flatpak run <id>`
Desktop entries	Host data dirs, `/run/host/usr/share`	Category Game; skips NoDisplay/Hidden and entries of known launchers; `TryExec` checked on the host	App icon on a padded card	`gio launch <file>`
Manual	–	User-entered, `imported_<n>` IDs	User-picked	User shell command

Duplicates and removal. The source-prefixed `game_id` is the only dedupe key; there is no fuzzy matching across sources beyond skipping known launchers' own desktop entries. With "Remove Uninstalled Games" on, any imported game not seen in a run is marked `removed` and its files are deleted on the next load.

Rewrite. Keeps imported, Steam, Lutris, Heroic, itch, Legendary and desktop sources (drops Bottles, RetroArch, Flatpak) and reads sources live from disk at every start instead of running an import step.

Data model, storage and launching

Each game is one JSON file named by its source-prefixed ID, with a matching cover file; settings live in GSettings, and launches go to the host through `flatpak-spawn --host`.

Game files. `$XDG_DATA_HOME/cartridges/games/<game_id>.json` (in the Flatpak this resolves under `~/.var/app/page.kramo.Cartridges/data/`). Format spec: `docs/game_id.json.md`.

Field	Type	Meaning
`game_id`	string	`<source>_<id>`, for example `heroic_gog_123`
`source`	string	Importer key, for example `heroic_gog`
`name`, `developer`	string	Title; developers joined with ", "
`executable`	string (legacy: argv list)	Shell command run on the host
`added`, `last_played`	unix seconds	0 means never played
`hidden`, `removed`, `blacklisted`	bool	Hidden by user; gone from source; confirmed non-game
`version`	number	Spec version (1.5 in stable code, 2.0 in rewrite)

Covers. `$XDG_DATA_HOME/cartridges/covers/<game_id>.tiff` (or `.gif` when animated), resized to 200×300 times the largest monitor scale factor. Images far from 2:3 get a blurred-background composite; icons are drawn at 0.7 scale.

Settings. GSettings schema `page.kramo.Cartridges`: `auto-import`, `exit-after-launch`, `cover-launches-game`, `high-quality-images`, `remove-missing`, a bool and location string per source, Heroic and Lutris sub-toggles, and SteamGridDB keys including `sgdb-key`, the API key stored in plain GSettings. A `.State` schema holds window size, sort mode and sidebar state. The rewrite schema keeps only `collections` (`aa{sv}`: name, icon, game IDs, removed).

Launching.

- Inside the Flatpak, `run_executable()` runs `flatpak-spawn --host /bin/sh -c <executable>` with cwd `$HOME` in a new session. This needs `--talk-name=org.freedesktop.Flatpak`, which is effectively a full sandbox escape.
- Store games open through host `xdg-open` with the store URI; `last_played` is saved at launch; "exit after launch" quits the app.
- Environment variables are not forwarded through `flatpak-spawn` (issue #362), so the Steam overlay can go missing; `steam://rungameid` also ignores Steam launch-option choices (issue #413 suggests `steam://launch/<id>/dialog`).
- There is no play-time tracking and no process tracking after launch.

Desktop integration.

- CLI: `--search TERM` and `--launch GAME_ID` (launches without opening the window).
- GNOME Shell search provider: a separate Python D-Bus service `page.kramo.Cartridges.SearchProvider` implementing `org.gnome.Shell.SearchProvider2`; it re-parses the JSON files itself, and issue #410 reports games with covers not appearing.
- The rewrite adds D-Bus activation; there is no custom D-Bus API.

Flatpak packaging and permissions

The shipped Flatpak takes read-write host access and the Flatpak session bus name; the rewrite narrows host access to read-only and adds gamepad input.

Permission	Stable v2.13.1	Rewrite branch	Why
`--share=network`, `--share=ipc`	Yes	Yes	SteamGridDB, Steam Store API
`--socket=wayland`, `--socket=fallback-x11`, `--device=dri`	Yes	Yes	Display, GPU
`--talk-name=org.freedesktop.Flatpak`	Yes	Yes	`flatpak-spawn --host` for every launch

Permission	Stable v2.13.1	Rewrite branch	Why
`--filesystem=host`	Read-write	`host:ro`	Read other launchers' data, check executables
`~/.var/app/<launcher>` paths, read-only	Steam, Lutris, Heroic, Bottles, itch, RetroArch	Steam, Heroic, itch	Other Flatpak launchers' data
`/var/lib/flatpak/{app,exports}`, `xdg-data/flatpak/{app,exports}`	Read-only	Read-only	Flatpak game discovery
`--device=input`	No	Yes	Gamepads via libmanette

The explicit `~/.var/app` paths alongside `host` suggest `host` does not expose other apps' sandbox data (unconfirmed). No Flathub reviewer discussion of these permissions was found. Flathub manifest: flathub/page.kramo.Cartridges.

Architecture patterns worth reusing and known gaps

The reusable value is in four patterns: a location resolver, sources as list models, a cover widget, and libmanette focus navigation. The README calls the `main` branch poor quality and asks forks not to reuse its code, so these are patterns to re-implement, not code to copy.

Patterns

1. **Location resolver** (stable `importer/location.py`): a GSettings key, an ordered list of candidate roots (native, then Flatpak), and required sub-paths. The first candidate with every sub-path wins and is saved; the user can override with a folder chooser. Simple and testable.
2. **Source as iterator**: a source yields games plus extra data (local cover path, icon path, online cover URL, Steam app ID). Per-entry errors are swallowed; user-facing errors are collected.
3. **Per-game manager pipeline** (stable `store/pipeline.py`): managers declare dependencies, run blocking or async in `Gio.Task` threads with a `Gio.Cancellable`, and retry; import progress derives from pipeline progress.

4. **Rewrite model stack**: each source is a `Gio.ListModel` of games inside a `Gio.ListStore` → `Gtk.FlattenListModel` → `FilterListModel` (drop removed and blacklisted) → `SortListModel` (locale collation, ignores a leading "The") → `Gtk.GridView` with a builder factory. Filters are declared in Blueprint and bound to window properties.
5. **Game as action group**: each game is a GObject with properties and a `Gio.SimpleActionGroup`; hide, unhide and remove enablement is bound with `Gtk.Expression`, and undo toasts hold an undo closure.
6. **Cover widget** (rewrite `ui/cover.py`): an `Adw.Bin` with a layout manager fixed at 200×300; `COVER` fit when within ±0.1 of 2:3, otherwise `CONTAIN` with a dimmed icon placeholder; GIF covers animate through a custom paintable.
7. **Gamepad** (rewrite `gamepads.py`): `Manette.Monitor` for hotplug; buttons map to activate, back and focus search; D-pad and left stick move focus with a 0.5 deadzone and 280 ms repeat.
8. **Rate limiter**: a token bucket for the Steam Store API with history persisted in GSettings.

Gaps users report

- Unmaintained; 43 open issues frozen; v2.13.1 crashes on GNOME 50 (#459).
- Launch only through other launchers: no per-game Proton, Wine, env vars or launch options.
- No play-time tracking, launch counts, descriptions or genres (#457, #359).
- Stable has no collections and no gamepad mode; the rewrite's edits are never saved because `Game.save()` is never called.
- No owned-but-not-installed games (#368) and no Faugus importer (#444).
- Imports are manual or at startup, with no file watching.
- Broad sandbox holes; the SteamGridDB key sits in plain GSettings.
- Bugs: Delete removes a game while typing (#337); animated covers make games disappear (#411).

Adopt, adapt, avoid

We should take Cartridges' importer knowledge and the rewrite's architecture, and fix its launch, sandbox and persistence gaps.

Adopt

- 2:3 covers at 200×300 logical size, fit-or-fill logic and the icon-on-blur fallback.
- Source-prefixed game IDs and a tombstone (`removed`) so deleted games are not re-imported.
- The location resolver: native and Flatpak candidates, saved result, user override.
- URI-handler launching for stores, using `steam://launch/<id>/dialog` where Steam launch options matter.
- Undo toasts for hide and remove.
- The rewrite's list-model → `GridView` stack, `Adw.Sidebar` with sources and collections, and libmanette focus navigation.
- A `SearchProvider2` service and a `--launch <id>` CLI.
- Heroic sub-source parsing, Lutris copy-then-query, and offline Steam `appinfo.vdf` parsing.

Adapt

- Narrow file access to read-only per-launcher paths instead of `--filesystem=host`.
- Keep host spawning but pass per-game environment and launch options explicitly.
- Store the SteamGridDB key with libsecret, not GSettings.
- Watch source files with `Gio.FileMonitor` for automatic sync.
- Add play-time tracking, a Faugus importer and the fullscreen mode on top of the gamepad code.
- Have the search provider call the app's own model over D-Bus instead of re-parsing files.

Avoid

- Copying code from `main`.
- `Gtk.FlowBox` with one widget per game.
- Online calls (Steam Store API) on the import path.
- A modal, non-closable import dialog.
- Unscoped Delete shortcuts.

Open questions

- Has any community fork picked up the rewrite branch, and could we collaborate rather than start fresh?
- Why was Cartridges dropped from GNOME Circle?
- What minimum libadwaita version does the rewrite need?

Sources

- kra-mo/cartridges (GitHub mirror; main and rewrite branches)
- main: data/gtk/window.blp, game.blp, preferences.blp, details-dialog.blp
- GSettings schema, docs/game_id.json.md
- importer/, store/, run_executable.py, search-provider/
- Devel Flatpak manifest
- rewrite branch
- Codeberg kramo/cartridges (archived) and open issues: #362, #410, #413, #457
- flathub/page.kramo.Cartridges (manifest and EOL flag)
- Flathub app page
- apps.gnome.org/Cartridges and GNOME Circle list
- kramo.page blog index; "Rewriting Cartridges" post dated 2025-12-09
- It's FOSS: Cartridges
- GamingOnLinux 2023 article
- LinuxPhoneApps: Cartridges